Multi-Stage Training Strategy for Small Rocket Detection with YOLO26s-P2 and COCO Hard Negative Mining

Team #3, SpaceY
Zifan Si
Jianqing Liu
Mike Chen
Xiaotian Lou

March 2026

Abstract

We present a systematic, multi-stage training pipeline for detecting small rockets in surveillance imagery captured by the RoCam system. Starting from a vanilla YOLO26S baseline ($mAP_{50\text{-}95} = 0.694$), we upgrade to the YOLO26S-P2 architecture—which adds a stride-4 detection head tailored for small targets—and introduce COCO hard negative mining to suppress false positives. Although the architectural change and harder training distribution initially *degrade* $mAP_{50\text{-}95}$ to 0.614 (-11.5%), a carefully designed three-stage fine-tuning protocol progressively recovers and surpasses the baseline, achieving a final $mAP_{50\text{-}95} = 0.750$ ($+8.1\%$ over the original) and $mAP_{50} = 0.958$. All experiments are conducted on a 4×NVIDIA H100 PCIe cluster using Distributed Data Parallel (DDP) training with mixed precision. We analyse each stage's contribution, discuss failure modes encountered during the process, and provide actionable insights for practitioners training small-object detectors.

Revision History

Date	Version	Notes
2026-03-01	1.0	Initial Draft
2026-03-03	2.0	Added experimental results and analysis

Contents

1 Symbols, Abbreviations, and Acronyms

Symbol	Description
mAP	Mean Average Precision — object detection evaluation metric
mAP_{50}	mAP at IoU threshold 0.50
$mAP_{50\text{-}95}$	mAP averaged across IoU thresholds 0.50 to 0.95
YOLO	You Only Look Once — real-time object detection architecture
COCO	Common Objects in Context — large-scale object detection dataset
DDP	Distributed Data Parallel — multi-GPU training method
SGD	Stochastic Gradient Descent — optimization algorithm
AdamW	Adam optimizer with weight decay
GFLOPs	Giga Floating Point Operations — computational complexity measure
P2 head	Stride-4 detection head in YOLO architecture
FT	Fine-Tuning

2 Introduction

Detecting small, fast-moving rockets in surveillance footage poses unique challenges for modern object detectors. The targets of interest typically occupy fewer than 30×30 pixels in 960×544 frames, can appear at *any* orientation (0–360°), and are subject to imaging degradation including motion blur, compression artefacts, and varying illumination.

The YOLO family of detectors (Ultralytics, 2025) offers an attractive balance between speed and accuracy for real-time deployment. However, vanilla YOLO architectures are optimised for objects at strides $\{8, 16, 32\}$, which may miss very small targets. The **P2 variant** introduces an additional detection head at stride 4, quadrupling the spatial resolution of the finest feature map.

In this report we document the complete training journey—from an initial YOLO26S baseline through architecture upgrade, data engineering (COCO hard negative mining), and a three-stage fine-tuning protocol. Our contributions are:

1. **Architecture upgrade**: adoption of YOLO26S-P2 with a stride-4 detection head, increasing small-target sensitivity at a modest +22% GFLOPs cost.

2. **COCO hard negative mining**: automated injection of 4,000 background images from MS COCO (Lin et al., 2014) to suppress false positives, constituting ∼25% of the training set.

3. **Multi-stage fine-tuning**: a three-stage protocol (close-mosaic, ultra-fine polish) that recovers the performance drop caused by the harder training distribution and pushes $\text{mAP}_{50\text{-}95}$ from 0.614 to 0.750.

4. **Failure analysis**: documentation of two failed fine-tuning attempts and the lessons learned (optimizer pitfalls, learning-rate scheduling).

3 Dataset

3.1 ROCAM Dataset

The ROCAM dataset is a single-class (`rocket`) object detection dataset derived from surveillance camera footage. Key statistics are summarised in Table 1.

Images are sourced from industrial surveillance cameras with a native resolution of 960×544 (non-square, ∼ 16:9 aspect ratio). Annotations follow the YOLO format (class, x_c, y_c, w, h normalised to $[0, 1]$).

3.2 COCO Hard Negative Mining

A persistent issue in small-object detection is the high false-positive rate: the model learns to fire on textured background regions that resemble the target at small scales. To address this, we inject **pure background images** from MS COCO (Lin et al., 2014) into the training set.

Table 1: ROCAM dataset statistics.

Split	Images	Resolution	Notes
Train (original)	11,832	960×544	Annotated rockets
Train (+ COCO neg)	15,832	mixed	+4,000 background images
Validation	3,378	960×544	
Test	3,421	960×544	
Total	**22,631**		Single class

Algorithm 1 COCO Hard Negative Mining Pipeline

Require: COCO train2017 + val2017 images (∼123K total)
Require: Target count $N = 4{,}000$
1: Download and extract COCO images (cached after first run)
2: $S \leftarrow \textsc{RandomSample}(\text{all COCO images}, N, \text{seed} = 0)$
3: **for** each image $s_i \in S$ **do**
4: Copy s_i to `images/train/coco_neg_{i}.jpg`
5: Create empty label file `labels/train/coco_neg_{i}.txt`
6: **end for**

The empty label files ensure YOLO treats these images as containing *zero* objects, providing pure background supervision. At $N = 4{,}000$, the negatives constitute ∼25.3% of the training set—a ratio chosen through iterative experimentation (see Section 8).

4 Model Architecture

4.1 YOLO26s Baseline

YOLO26 (Ultralytics, 2025) is the latest generation of the Ultralytics YOLO family, featuring an updated backbone and a dual-head design (one-to-one for end-to-end inference, one-to-many for higher-recall training). The **small** variant (YOLO26S) has ∼10M parameters and natively detects at strides $\{8, 16, 32\}$.

4.2 P2 Variant: Stride-4 Detection Head

The YOLO26S-P2 variant adds a **P2 detection head** at stride 4, which operates on a feature map with 4× the spatial resolution of the standard finest level. This is critical for our application where rocket targets can be as small as 10×10 pixels.

Both variants use transfer learning from COCO-pretrained `yolo26s.pt` weights, with matching layers automatically migrated to the P2 architecture.

Table 2: Architecture comparison: YOLO26S vs. YOLO26S-P2.

Model	Parameters	GFLOPs	Detection Strides
YOLO26S	∼10.0M	baseline	$\{8, 16, 32\}$
YOLO26S-P2	∼9.8M	+22%	$\{4, 8, 16, 32\}$

5 Training Methodology

Our training pipeline consists of four distinct phases, as illustrated in Figure 1.

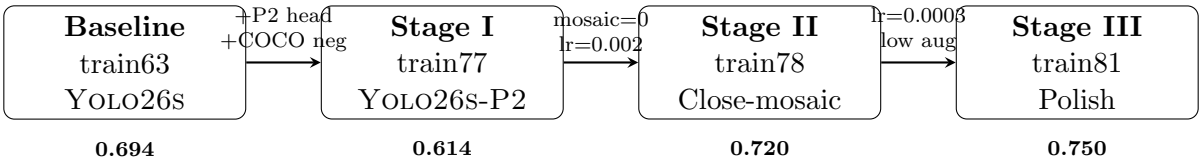


Figure 1: Training pipeline overview. Numbers below each stage indicate the best $\text{mAP}_{50\text{-}95}$ achieved. Note the initial *decrease* after Stage I, which is recovered through fine-tuning.

5.1 Baseline: Vanilla YOLO26s (train63)

The baseline model uses the vanilla YOLO26S architecture trained from COCO-pretrained weights on a single GPU. Key configuration:

- **Hardware**: 1× NVIDIA H100 PCIe (81 GB)
- **Batch size**: 64, image size $[544, 960]$
- **Epochs**: 420 (all completed, no early stopping)
- **Scheduler**: linear decay, $\text{lr}_0 = 0.01$, $\text{lrf} = 0.01$
- **Augmentation**: aggressive—mosaic= 1.0, cutmix= 0.1, scale= 0.3, close_mosaic= 80
- **COCO negatives**: none

Result: epoch 419, $\text{mAP}_{50} = 0.948$, $\text{mAP}_{50\text{-}95} = 0.694$.

5.2 Stage I: P2 Architecture + COCO Negatives (train76/77)

Stage I introduces two major changes simultaneously:

1. **Architecture**: upgrade from YOLO26S to YOLO26S-P2 (`yolo26s-p2.yaml`), adding the stride-4 head.
2. **Data**: inject COCO hard negatives (initially 1,000 in train76, increased to 4,000 in train77).

Augmentation is *reduced* relative to the baseline to protect small targets: mosaic $1.0 \to 0.8$, cutmix $0.1 \to 0.0$, scale $0.3 \to 0.15$.

Table 3: Stage I configuration evolution.

Parameter	**train76**	**train77**
GPUs	3 (H100)	4 (H100)
COCO negatives	1,000	4,000
Patience	30	50
lrf	0.01	0.02
close_mosaic	150	250
Best $\text{mAP}_{50\text{-}95}$	0.652 (ep335)	0.614 (ep267)
Epochs run	365/600	317/600

Critical issue. Both runs terminated via early stopping *before* the `close_mosaic` epoch was reached. In train77 with close_mosaic= 250, mosaic augmentation was scheduled to disable at epoch $600 - 250 = 350$, but training stopped at epoch 317 (patience= 50). Consequently, the model **never trained on full-resolution images**—it only ever saw $\frac{1}{4}$-scale mosaic composites, severely limiting small-target localisation precision.

5.3 Stage II: Close-Mosaic Fine-Tuning (train78)

To address the missed close-mosaic phase, we fine-tune from `train77/best.pt` (epoch 267) with mosaic *completely disabled*:

- **Mosaic**= 0.0, **mixup**= 0.0, cutmix= 0.0
- $\text{lr}_0 = 0.002$ ($\frac{1}{5}$ of Stage I), cosine schedule (Loshchilov and Hutter, 2016), $\text{lrf} = 0.05$
- 5-epoch warmup to smooth the distribution shift
- 100 epochs, patience= 40
- Erasing reduced $0.4 \rightarrow 0.2$

Result: The model saw full-resolution (960×544) images for the first time, and $\text{mAP}_{50\text{-}95}$ **jumped from** 0.614 **to** 0.720 (+17.3%), reaching $\text{mAP}_{50} = 0.955$. This is the **single largest improvement** in the entire pipeline.

5.4 Stage III: Ultra-Fine Polish (train81)

The final stage applies a very low learning rate to "polish" the model into its optimal basin:

- Resume from `train78/best.pt` (epoch 93, $\text{mAP}_{50\text{-}95} = 0.720$)
- $\text{lr}_0 = 0.0003$ ($\frac{1}{7}$ of Stage II), **SGD explicitly set** (not `auto`)
- Cosine decay to $\text{lr}_0 \times 0.2 = 6 \times 10^{-5}$

- `multi_scale = False`: fixed 960 px for deployment resolution
- Augmentation further reduced: scale 0.15 → 0.08, shear 5.0 → 2.0, translate 0.05 → 0.02
- Albumentations probabilities lowered by ∼30%
- 60 epochs, patience= 25

Result: $\text{mAP}_{50\text{-}95}$ improved from 0.720 to **0.750** (+4.2%), with $\text{mAP}_{50} = 0.958$, precision = 0.961, recall = 0.915.

5.5 Failed Attempts and Lessons Learned

train79 (1 epoch only). An attempt to fine-tune with `optimizer = ''auto''` and $\text{lr}_0 = 0.0003$. Ultralytics' auto-optimizer selected AdamW and *ignored* the specified lr_0, applying its own scaling. The run was aborted after one epoch upon discovering the unexpected learning rate.

train80 (no results). Fixed the optimizer to `SGD` but encountered a configuration issue that prevented results from being saved.

Lesson. When fine-tuning with a specific learning rate, **always set the optimizer explicitly** (e.g., `optimizer='SGD'`). The `auto` mode applies internal heuristics that can override user-specified hyperparameters.

6 Data Augmentation Strategy

A distinctive aspect of this pipeline is the **progressive reduction** of augmentation intensity across stages, summarised in Table 4.

180° rotation. Full rotation is retained across *all* stages because rockets in flight can appear at any orientation—this augmentation matches the true data distribution.

Albumentations integration. We inject a custom imaging-degradation pipeline via a monkey-patch on Ultralytics' `Albumentations` class, ensuring DDP compatibility (Buslaev et al., 2020). The pipeline includes:

- **MotionBlur** ($k \leq 7$, p=0.15): simulates camera/target motion
- **GaussianBlur** ($k \leq 7$, p=0.10): generic defocus
- **GaussNoise** ($\sigma \in [0.01, 0.03]$, p=0.15): sensor noise
- **JPEG Compression** ($q \in [40, 95]$, p=0.20): compression artefacts
- **RandomBrightnessContrast** (p=0.20): illumination variation
- **RandomGamma** (p=0.10), **CLAHE** (p=0.10): dynamic range adjustments

In Stage III, all probabilities are reduced by ∼30% and kernel sizes decreased ($k \leq 5$) to minimise perturbation during the polishing phase.

Table 4: Augmentation parameters across training stages. Values decrease from left to right, reflecting the shift from exploration (learning diverse features) to exploitation (precision refinement).

Parameter	**Baseline** (train63)	**Stage I** (train77)	**Stage II** (train78)	**Stage III** (train81)
mosaic	1.0	0.8	0.0	0.0
mixup	0.10	0.10	0.0	0.0
cutmix	0.10	0.0	0.0	0.0
scale	0.30	0.15	0.15	0.08
shear	5.0	5.0	5.0	2.0
translate	0.05	0.05	0.05	0.02
perspective	0.0002	0.0002	0.0002	0.0001
degrees	180	180	180	180
flipud	0.5	0.5	0.5	0.5
fliplr	0.5	0.5	0.5	0.5
erasing	0.4	0.4	0.2	0.05
hsv_h / s / v	.015/.6/.4	.015/.6/.4	.015/.6/.4	.01/.4/.3

7 Experimental Results

7.1 Overall Performance Summary

Table 5 presents the best metrics for each training stage.

Table 5: Best validation metrics across all training stages.

Stage	**Run**	**Model**	**Best Ep.**	**Precision**	**Recall**	**mAP$_{50}$**	**mAP$_{50\text{-}95}$**
Baseline	train63	YOLO26S	419	0.945	0.897	0.948	0.694
Stage I (v1)	train76	YOLO26S-P2	335	0.918	0.886	0.935	0.652
Stage I (v2)	train77	YOLO26S-P2	267	0.917	0.857	0.920	0.614
Stage II	train78	YOLO26S-P2	93	0.955	0.912	0.955	0.720
Stage III	train81	YOLO26S-P2	52	**0.961**	**0.915**	**0.958**	**0.750**

7.2 Training Curves

Figure 2 shows the mAP$_{50\text{-}95}$ progression across all stages, clearly illustrating the initial degradation and subsequent recovery.

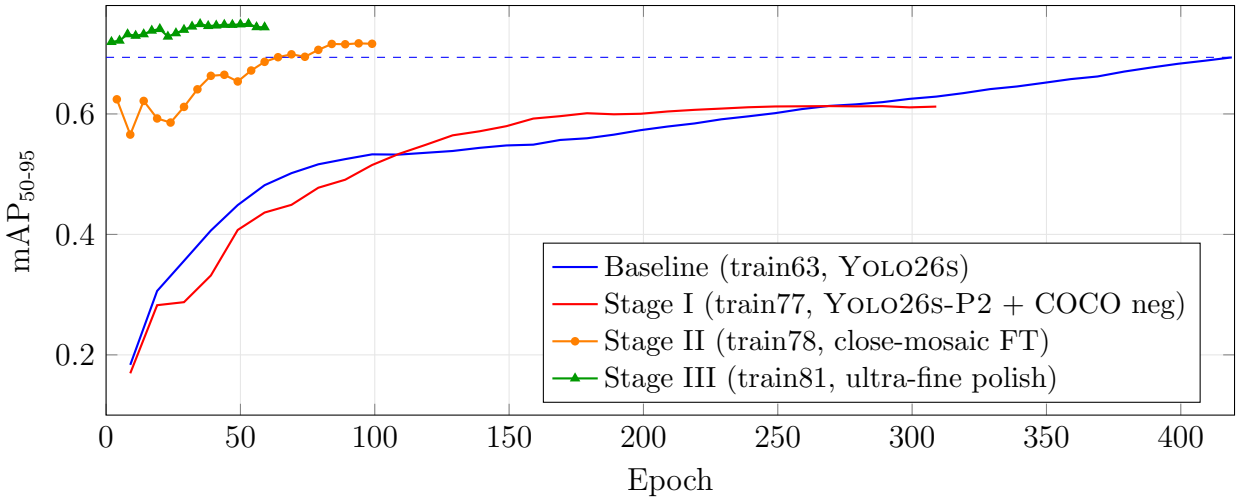


Figure 2: $\text{mAP}_{50\text{-}95}$ learning curves across all stages. The dashed blue line marks the baseline best (0.694). Stage II and III are plotted on their own epoch axes. Stage II shows the dramatic recovery once mosaic is disabled.

7.3 Validation Loss Curves

Figure 3 compares the validation box loss between the baseline and Stage I, revealing the convergence pattern.

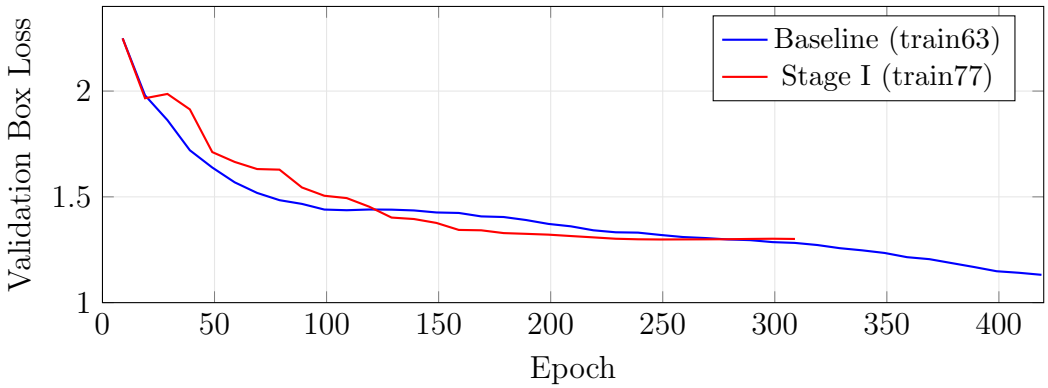


Figure 3: Validation box loss comparison. The baseline (train63, YOLO26S) continues to decrease throughout 420 epochs, while Stage I (train77, YOLO26S-P2) *stagnates* around epoch 250 and begins to *slightly increase*—a sign that the model benefits from distribution shift (disabling mosaic) rather than more of the same augmented data.

7.4 Stage-by-Stage Improvement Breakdown

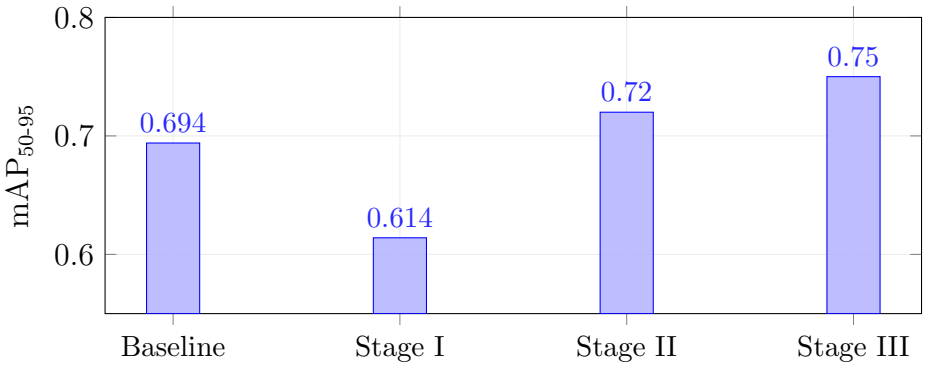
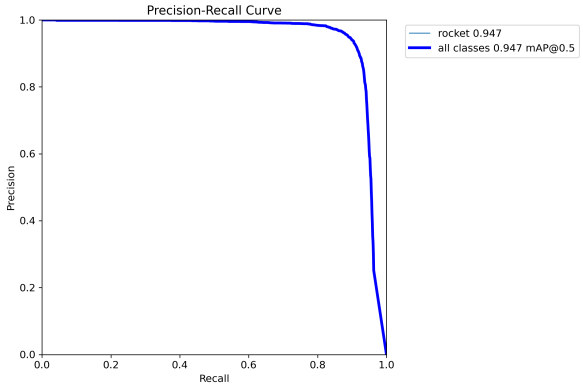


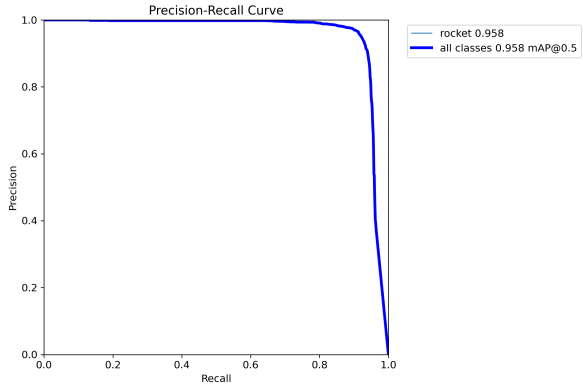
Figure 4: $\text{mAP}_{50\text{-}95}$ progression across training stages. The initial drop from Baseline to Stage I is due to the harder training distribution (COCO negatives + mosaic-only training). Stages II and III progressively recover and exceed the baseline.

7.5 Precision–Recall Curves

Figure 5 shows the precision–recall curves for the baseline and the final model, generated by Ultralytics.



(a) Baseline (train63): $\text{mAP}_{50} = 0.948$

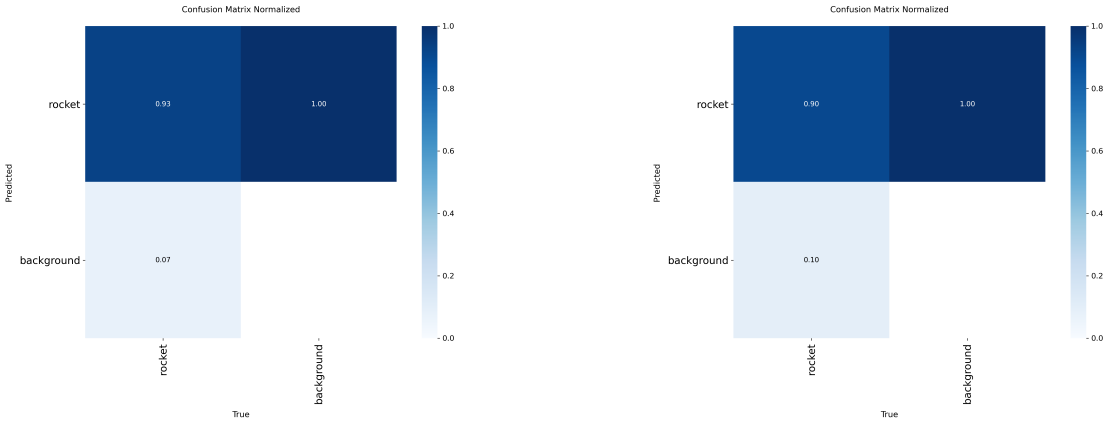


(b) Final model (train81): $\text{mAP}_{50} = 0.958$

Figure 5: Precision–Recall curves for the baseline and final model. The final model achieves both higher precision and higher recall.

7.6 Confusion Matrices

Figure 6 presents the normalised confusion matrices, showing the reduction in false negatives and false positives.

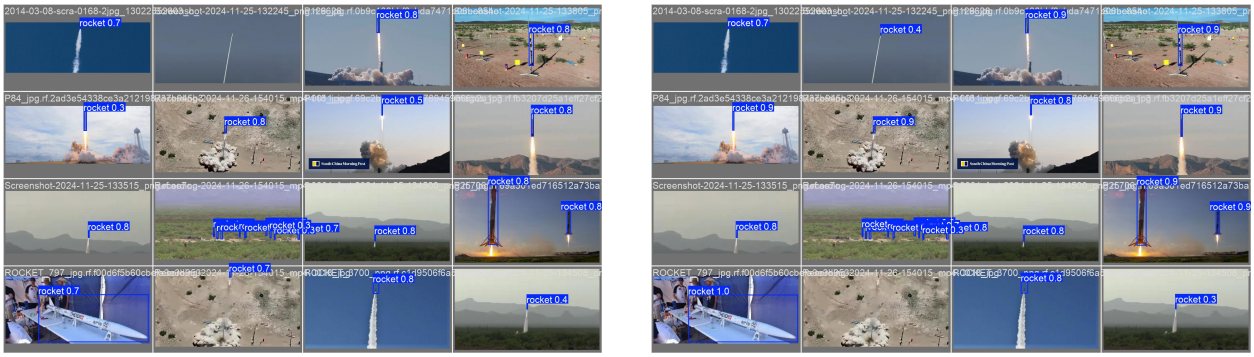


(a) Baseline (train63) (b) Final model (train81)

Figure 6: Normalised confusion matrices. The final model shows improved true-positive rate and reduced background confusion.

7.7 Qualitative Results

Figure 7 provides a visual comparison of detection quality on validation images.



(a) Baseline predictions (train63) (b) Final model predictions (train81)

Figure 7: Validation batch predictions. The final model produces tighter bounding boxes and higher confidence scores, particularly on small targets.

8 Analysis and Discussion

8.1 Why Did Stage I Degrade Performance?

The $\text{mAP}_{50\text{-}95}$ drop from 0.694 (baseline) to 0.614 (Stage I) is attributable to **three compounding factors**:

1. **COCO negative dilution**: Adding 4,000 background images (25% of training data) forces the model to learn a harder decision boundary, trading recall for false-positive

suppression. While beneficial for deployment (fewer false alarms), this manifests as lower $\text{mAP}_{50\text{-}95}$ on the validation set that contains no COCO images.

2. **Mosaic starvation**: Mosaic augmentation (Bochkovskiy et al., 2020) composites four images into one, reducing each to $\frac{1}{4}$ resolution. For our already-small rockets, this pushes targets below the detection threshold. The intended remedy—disabling mosaic in the final epochs (`close_mosaic`)—was *never activated* because early stopping triggered first.

3. **Architectural cold start**: The P2 head introduces new layers initialised from transferred weights that only partially match. These require extended training to specialise for the target domain, particularly at the finest scale.

8.2 The Close-Mosaic Timing Problem

The interaction between `close_mosaic` and `patience` (early stopping) is a subtle but critical failure mode:

$$\text{mosaic\_off\_epoch} = \text{total\_epochs} - \text{close\_mosaic} = 600 - 250 = 350$$

But early stopping triggered at epoch 317 (< 350), so the transition never occurred. The solution is to decouple the two by performing close-mosaic fine-tuning as a *separate training stage* with mosaic explicitly set to 0, which is precisely what Stage II achieves.

Design recommendation. For long training runs with large `close_mosaic` windows, either (a) set patience $>$ epochs $-$ close_mosaic_epoch, or (b) implement close-mosaic as a separate fine-tuning stage.

8.3 Effect of COCO Negative Count

We experimented with two levels of COCO negative injection:

Table 6: Impact of COCO negative sample count.

Run	**COCO Negatives**	**% of Training Set**	**$\text{mAP}_{50\text{-}95}$**
train63 (baseline)	0	0%	0.694
train76	1,000	7.8%	0.652
train77	4,000	25.3%	0.614

More negatives produce a harder training problem (lower $\text{mAP}_{50\text{-}95}$) but are expected to improve *deployment* performance by suppressing false positives on unseen backgrounds. The 25% ratio was chosen as a compromise: enough to provide meaningful false-positive supervision without overwhelming the positive examples.

8.4 Diminishing Returns in Stage III

Stage III contributed a +4.2% improvement to $\text{mAP}_{50\text{-}95}$, compared to Stage II's +17.3%. This is expected: as the learning rate decreases (0.0003 vs. 0.002), the model makes increasingly small parameter updates. The $\text{mAP}_{50\text{-}95}$ curve in Figure 2 shows the Stage III plateau beginning around epoch 35, suggesting the model has effectively converged.

8.5 Optimizer Selection Pitfall

The failed train79 run highlights an important implementation detail in Ultralytics: when `optimizer=''auto''` is set, the framework may internally select AdamW and apply its own learning-rate scaling heuristics, effectively *overriding* the user-specified lr_0. For fine-tuning where precise learning rate control is essential, **always specify the optimizer explicitly**.

9 Conclusion

We have presented a systematic, multi-stage training pipeline for small rocket detection that progresses from a vanilla YOLO26S baseline ($\text{mAP}_{50\text{-}95} = 0.694$) to a final YOLO26S-P2 model achieving $\text{mAP}_{50\text{-}95} = 0.750$ and $\text{mAP}_{50} = 0.958$. The key insights are:

1. **Architecture and data changes must be coupled with training strategy.** The P2 head and COCO negatives initially degraded performance; only through staged fine-tuning was their potential unlocked.

2. **Close-mosaic is critical for small objects.** Training exclusively on mosaic-augmented images limits small-target localisation. Disabling mosaic via a dedicated fine-tuning stage produced the largest single improvement (+17.3% $\text{mAP}_{50\text{-}95}$).

3. **Progressive augmentation reduction** follows the principle of curriculum learning: heavy augmentation for feature diversity, then minimal augmentation for precision.

4. **Deployment considerations.** The final model runs at 960×544 resolution with ∼9.8M parameters, suitable for real-time inference on GPU-equipped edge devices.

The complete training pipeline—from COCO download to final polishing—is implemented in four Python scripts, enabling full reproducibility.

References

Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.

Ultralytics. Ultralytics YOLO: Real-time object detection. https://github.com/ultralytics/ultralytics, 2025.

A Hardware Configuration

Table 7: Training infrastructure.

Component	**Specification**
GPU	4× NVIDIA H100 PCIe, 81 GB HBM3 each
Training mode	Distributed Data Parallel (DDP), NCCL backend
Precision	Mixed (AMP, FP16/FP32)
Batch size	192 total (48 per GPU)
Disk cache	Enabled (`cache='disk'`)
Workers	8 per process

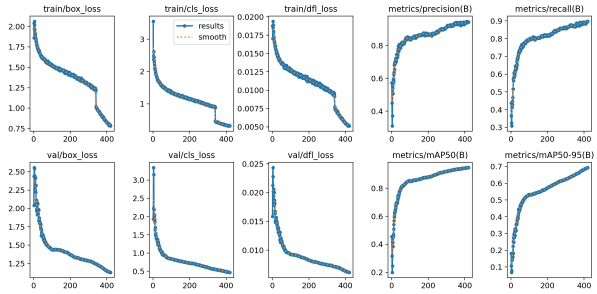
B Complete Hyperparameter Table

Table 8: Full training hyperparameters across all stages.

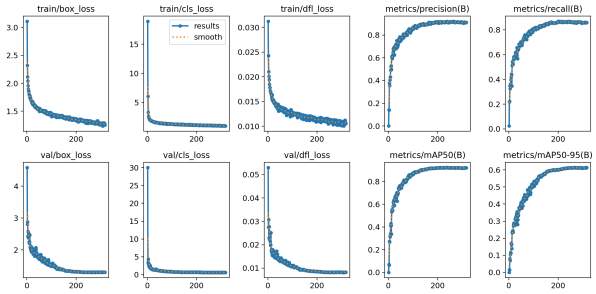
Hyperparameter	**Baseline** (train63)	**Stage I** (train77)	**Stage II** (train78)	**Stage III** (train81)
Model	yolo26s.pt	yolo26s-p2.yaml	(from train77)	(from train78)
GPUs	1	4	4	4
Batch	64	192	192	192
Image size	544×960	960	960	960
Epochs	420	600	100	60
Patience	30	50	40	25
Optimizer	auto	auto	auto	SGD
lr_0	0.01	0.01	0.002	0.0003
lrf	0.01	0.02	0.05	0.2
Schedule	linear	linear	cosine	cosine
Warmup (ep)	3	3	5	2
multi_scale	True	True	True	False
COCO neg.	0	4,000	4,000	4,000
Best $\mathbf{mAP}_{50\text{-}95}$	**0.694**	**0.614**	**0.720**	**0.750**

C Training Results Plots (Ultralytics)

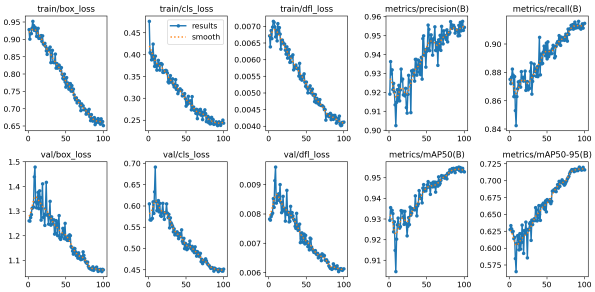
Figure 8 shows the built-in Ultralytics training results plots for each stage.



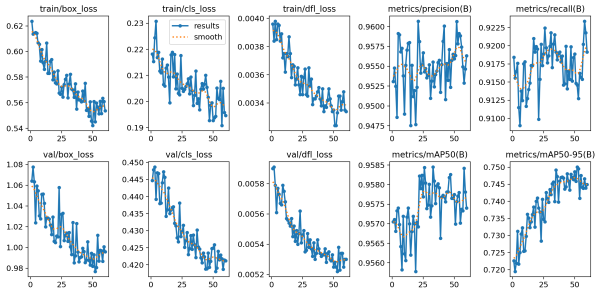
(a) Baseline (train63, 420 epochs)



(b) Stage I (train77, 317 epochs)



(c) Stage II (train78, 100 epochs)



(d) Stage III (train81, 60 epochs)

Figure 8: Ultralytics auto-generated training result plots for each stage, showing train/val losses, precision, recall, and mAP metrics.